

Functions as values: iterators and streams

Block 1 — Iterators: who drives the traversal?

SW7 · Aalborg University Copenhagen · Autumn 2026

Three questions from session 4

What does a machine need to call a function?

A frame: it holds the work still to do after the call.

Do we need a loop construct?

No: the loop is a value, `fold`.

In which order, and when, are arguments evaluated?

The language decides, unless we wrap an argument in a function.

implicit: the machine keeps the work still to do, in its frames

```
def count(m):  
    if m[0] == "Note": return 1  
    return count(m[1]) + count(m[2])
```

PYTHON

explicit: the program keeps it, in a stack of its own

```
def count(m):  
    n, stack = 0, [m]  
    while stack:  
        m = stack.pop()  
        if m[0] == "Note": n += 1  
        else: stack += [m[2], m[1]]  
    return n
```

PYTHON

the body of the loop is an argument, `f`

```
let rec fold f acc = function  
| [] -> acc  
| x :: xs -> fold f (f acc x) xs  
  
let sum = fold (fun acc x -> acc + x) 0
```

OCAML

each branch is wrapped in a function

```
let my_if c t e = if c then t () else e ()  
  
let rec fact n =  
    my_if (n = 0) (fun () -> 1)  
    (fun () -> n * fact (n - 1))
```

OCAML

For answers 2 and 3 we passed a **function as a value**. In session 6 we do the same for answer 1, with **continuations**.

Today

We use **functions as values** to answer how a program itself can control, step by step, a traversal of its data, possibly infinite, and its own computation: computing a value only when it is needed.

	the question	the function is
Block 1	Iterators: who drives the traversal?	one step of a traversal
Block 2	Thunks and streams: when is a value computed?	a computation not yet run

Warm-up. Questions on paper, then exercises in OCaml, on blocks 1 and 2.

Lab. Making a mini text editor, on a library that we write in three parts; the third goes with session 6.

Block 1: the five parts

- | | | |
|---|--------------------------|---|
| 1 | Closures | We start with what a function value is: a function with its environment, which can keep a state between two calls. |
| 2 | Traversing a list | First, we traverse a list by giving it a function, which it calls on every element (the list pushes the elements: internal iteration). |
| 3 | Stopping | Problem: in a strict language, the answer of this function cannot stop the traversal before the last element. |
| 4 | Iterators | Second, we traverse by asking for one element at a time (we pull the elements: external iteration). Using closures, the iterator remembers where it is in the list between two calls. |
| 5 | Generators | We let the language write the iterator: we write a function with <code>yield</code> , and the language makes the object that remembers where the function stopped. |

Closures

What does a function value carry?

A function value is code and an environment

```
let add n x = x + n
```

OCAML

```
let add3 = add 3
```

```
add3 10      (* 13 *)
```

The definition of `add` names two arguments, and we give it one, `3`. What is `add3`?

the value `add3`

the value <code>add3</code>	
code	<code>fun x -> x + n</code>
environment	<code>n = 3</code>

Lab 4's evaluator builds this value for every function:

```
| Closure of string * term * env
```

OCAML

A function value of this kind is a **closure**.^{*} `add` is **curried**: it takes `n` and returns a function. `add 3` is a **partial application**.

^{*}The name and the construction are Landin's (1964): the function is closed by its environment.

A function value is code and an environment

```
let add n = fun x -> x + n
```

OCAML

```
let add3 = add 3
```

```
add3 10          (* 13 *)
```

The call `add 3` returns `fun x -> x + n`. Later, the call `add3 10` still finds `n = 3`: the function carries it.

the value `add3`

code

```
fun x -> x + n
```

environment

```
n = 3
```

Lab 4's evaluator builds this value for every function:

```
| Closure of string * term * env
```

OCAML

A function value of this kind is a **closure**.^{*} `add` is **curried**: it takes `n` and returns a function. `add 3` is a **partial application**.

^{*}The name and the construction are Landin's (1964): the function is closed by its environment.

Three closures made in a loop, with var

```
var fs = [];  
for (var i = 0; i < 3; i++) fs.push(() => i);  
console.log(fs.map(f => f()));
```

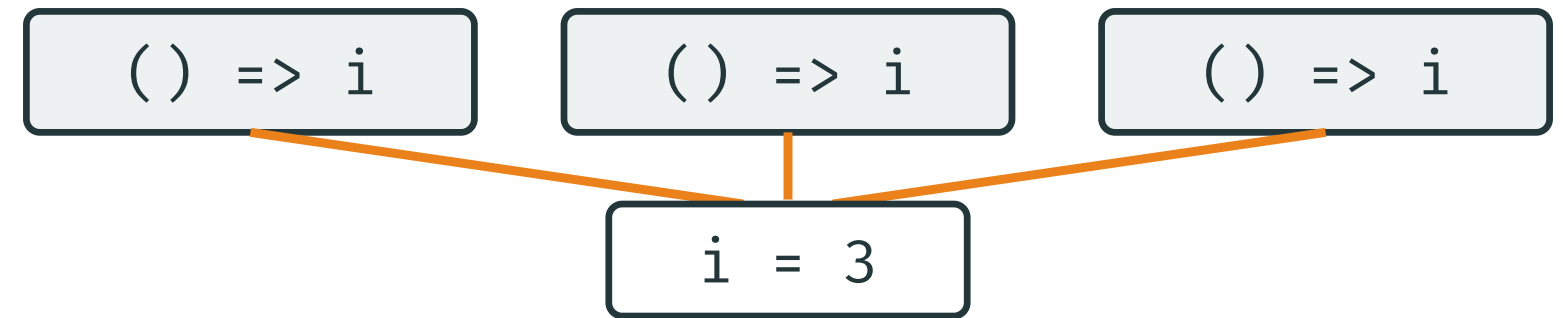
JAVASCRIPT

Run Reset

[3, 3, 3]

The loop makes three functions. We call them after the loop. What does the program print?

`var i`: one variable for the whole loop



In OCaml, the one variable is a `ref`

```
let fs = ref [] and i = ref 0  
let () =  
  while !i < 3 do  
    fs := (fun () -> !i) :: !fs;  
    incr i  
  done  
List.rev_map (fun f -> f ()) !fs
```

OCaml ↗

[3; 3; 3]

- `var i` makes **one variable**. The three functions keep that same variable.
- We call the functions after the loop. The loop has ended on `i = 3`, and each function reads 3.

A closure keeps the **variable**, not the value it held when the closure was made.

Three closures made in a loop, with `let`

```
var fs = [];  
for (let i = 0; i < 3; i++) fs.push(() => i);  
console.log(fs.map(f => f()));
```

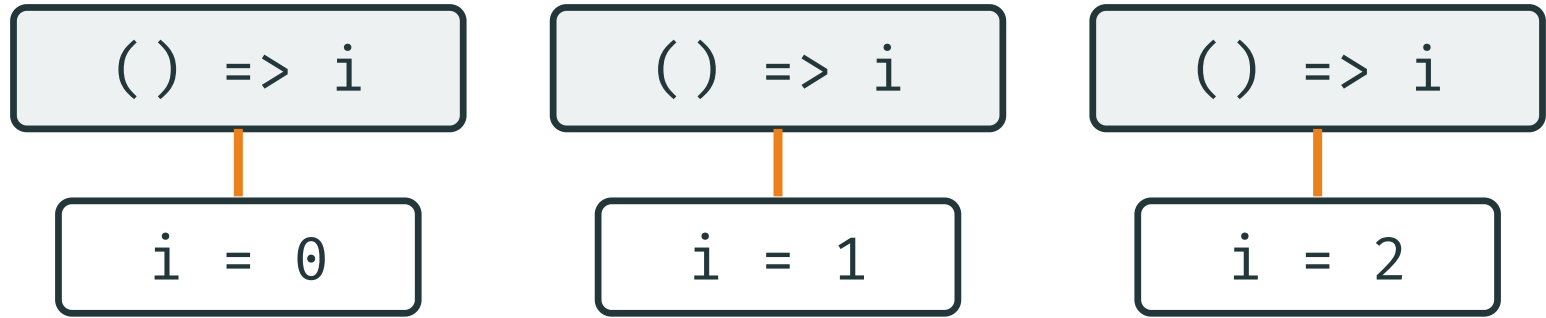
Run

Reset

```
[0, 1, 2]
```

The same program, with `let` in the place of `var`. What does it print?

`let i`: one variable per round of the loop



In OCaml, `for` makes a new `i` at every round

```
let fs = ref []  
let () =  
  for i = 0 to 2 do  
    fs := (fun () -> i) :: !fs  
  done  
List.rev_map (fun f -> f ()) !fs
```

Ocaml ↗

```
[0; 1; 2]
```

- `let i` makes **a new variable** at every round of the loop. Each function keeps its own.
- We call the functions after the loop, and they read 0, 1 and 2.

The rule has not changed: a closure keeps its **variable**. With `let` there are three variables, and none of them changes.

Here the closures read their variable. What if a closure changes it?

A counter: a closure with a state of its own

```
let counter () =  
  let n = ref 0 in  
  fun () -> incr n; !n
```

OCaml

```
let a = counter ()  
let b = counter ()  
(* a (), then a (), then b () *)
```

1 2 1

```
const counter = () => {  
  let n = 0;  
  return () => ++n;  
};  
  
const a = counter(), b = counter();  
console.log(a(), a(), b());
```

JAVASCRIPT

Run

Reset

1 2 1

Every call of `counter` makes a new variable `n`: `a` and `b` do not share it.

The closure changes `n` itself, and no other code can read it or change it: a **state of its own**.

This is **encapsulation**. In this way an iterator remembers where it is in a list.

A closure and an object

```
const counter = () => {  
  let n = 0;  
  return { tick: () => ++n,  
           reset: () => { n = 0; } };  
};  
const c = counter();  
c.tick(); c.tick(); c.reset();  
console.log(c.tick());
```

JAVASCRIPT

Run

Reset

1

```
class Counter {  
  private int n = 0;  
  int tick() { return ++n; }  
  void reset() { n = 0; }  
}  
Counter c = new Counter();  
c.tick(); c.tick(); c.reset();  
System.out.println(c.tick());
```

Java ↗

1

A closure is an object with **one method**. An object is **several closures** over the same fields.

Sources: Leroy 2026, §3.3. Objects add to this the choice of the method at run time and inheritance: Cook 2009.

Traversing a list

How do we visit every element of a list?

Two traversals of a list, in Java

```
List<String> words = List.of("to", "be", "or", "not");
```

[Java ↗](#)

```
words.forEach(w ->  
    System.out.print(w + " "));
```

[JAVA](#)

to be or not

```
Iterator<String> it = words.iterator();  
while (it.hasNext()) {  
    String w = it.next();  
    System.out.print(w + " ");  
}
```

[JAVA](#)

to be or not

Two traversals of a list, in OCaml

```
let words = ["to"; "be"; "or"; "not"]
```

[OCaml ↗](#)

```
List.iter  
  (fun w -> print_string (w ^ " "))  
  words
```

OCAML

```
to be or not
```

```
let rec print = function  
  | [] -> ()  
  | w :: rest ->  
    print_string (w ^ " ");  
    print rest
```

OCAML

```
to be or not
```

Does the list push each element into our function, or do we pull each element out of the list?

Two folds on a list, from session 4

```
let rec fold_left f acc = function
  | [] -> acc
  | x :: xs -> fold_left f (f acc x) xs
```

OCaml

From the first element to the last, with an accumulator.

```
let rec fold_right f xs z =
  match xs with
  | [] -> z
  | x :: xs -> f x (fold_right f xs z)
```

OCaml

Every `::` is replaced by a call of `f`, and `[]` by `z`.

Both call `f` **once per element**. The recursion is written once, in the fold; we pass `f` and write no recursion.

map, filter and length, written with fold_right

```
let map f l      = fold_right (fun x r -> f x :: r) l []
let filter p l   = fold_right (fun x r -> if p x then x :: r else r) l []
let length l     = fold_right (fun _ n -> n + 1) l 0
```

OCaml ↗

```
let text = ['h'; 'i'; '\n'; 'y'; 'o']
```

OCAML

```
map Char.uppercase_ascii text
filter (fun c -> c <> '\n') text
length text
```

```
['H'; 'I'; '\n'; 'Y'; 'O']
['h'; 'i'; 'y'; 'o']
5
```

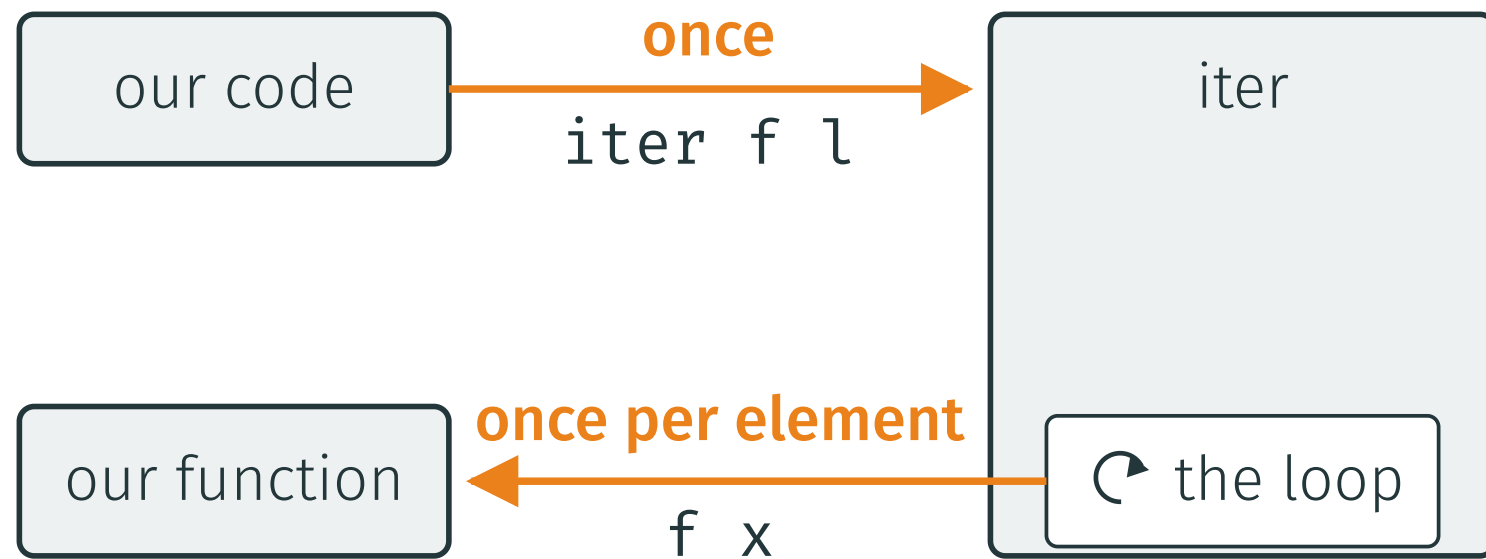
Each of the three uses **every element** of the list. None has a reason to stop before the last one.

With `fold_left`, `f x :: r` puts each element in front of those met before it: the list is reversed.

Internal iteration: the collection calls our function

```
let iter f l = fold_left (fun () x -> f x) () l
```

OCaml



We call `iter` once and pass it a function. `iter` calls the function: it decides when, and how many times.

OCaml

```
List.iter f xs
```

JavaScript

```
xs.forEach(f)
```

Java

```
xs.forEach(f)
```

This style is **internal iteration**: the loop is inside the list's own function. The library calls our code; the general name is **inversion of control**.*

* Johnson & Foote 1988; the name is discussed in Fowler 2005. The pattern: Gamma et al. 1994, *Iterator*.

Stopping

Can our function stop the traversal?

Is there a newline in the text?

```
let text = ['h'; 'i'; '\n'; 'y'; 'o']  
let visited = ref 0
```

OCaml ↗

```
let exists p l =  
  fold_left (fun found x ->  
    incr visited;  
    found || p x)  
    false l  
  
exists (fun c -> c = '\n') text
```

`exists` is one more fold. The function we pass is a closure: it uses `visited` and `p`.

The newline is the third element. What is `!visited` at the end?

`found=true visited=5`

`filter` needed every element. `exists` has its answer at the third one.

The function knew the answer at the third element and had no way to say so. Ignoring an element is **not stopping**.

Stopping: a different interface

```
type 'b go = Continue of 'b | Stop of 'b
```

OCaml

```
let rec fold_until f acc = function
  | [] -> acc
  | x :: xs ->
      match f acc x with
      | Stop a -> a
      | Continue a -> fold_until f a xs
```

```
let () = visited := 0
let exists p l =
  fold_until (fun _ x ->
    incr visited;
    if p x then Stop true
    else Continue false)
  false l
```

OCaml ↗

```
found=true visited=3
```

Our function's answer now carries a **decision**, `Stop` or `Continue`, and the loop reads it.*

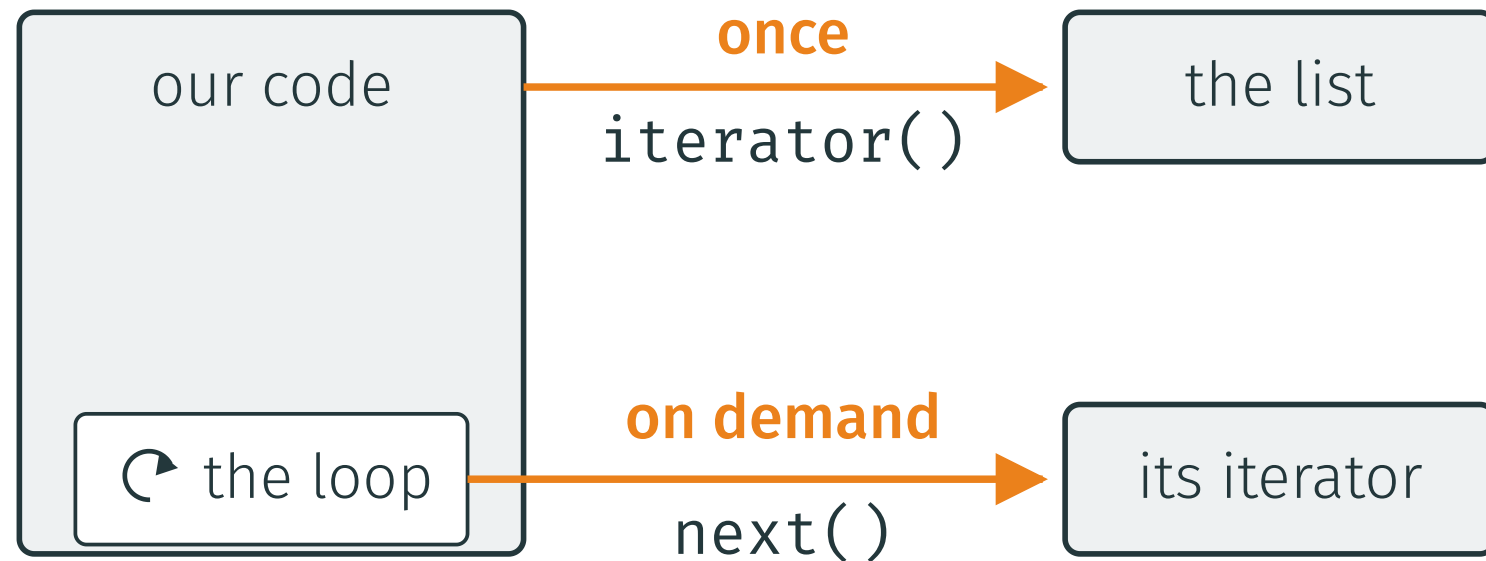
Our function can stop, but it runs only when `fold_until` calls it, and we do not choose when. In part 4 **we** call `next`, which settles both: we ask when we want, and we stop by not asking.

* After `fold_until` in OCaml's Base library. An exception also leaves a traversal: session 6.

Iterators

How can the consumer choose the next step?

External iteration: we ask for the next element



```
interface Iterator<T> {  
    boolean hasNext();  
    T next();  
}
```

JAVA

We ask the list for an iterator, once. We call `next`: we decide when, and how many times.

This style is **external iteration**: the collection gives us an **iterator**, and the consumer calls it.

Source: Gamma et al. 1994, *Iterator*. Java: `java.util.Iterator`, `java.lang.Iterable`.

The newline in the text, with an iterator

```
Iterator<Character> it = text.iterator();  
while (it.hasNext() && !found) {  
    visited++;  
    found = it.next() == '\n';  
}
```

Java ↗

```
found=true visited=3
```

The loop is ours. Between two calls of `next` the traversal waits, and we may stop asking.

```
for (char c : text) { ... }
```

JAVA

The `for` over a collection is this loop, written by the compiler: it calls `iterator`, then `hasNext` and `next`.

An iterator for a list, written by hand

```
class ListIterator<T>
    implements Iterator<T> {
    private Node<T> here;
    ListIterator(Node<T> l) { here = l; }

    public boolean hasNext() {
        return here != null;
    }
    public T next() {
        if (here == null)
            throw new NoSuchElementException();
        T x = here.value();
        here = here.next();
        return x;
    }
}
```

Java ↗

```
let iterator l =
  let here = ref l in
  fun () ->
    match !here with
    | [] -> None
    | x :: rest ->
        here := rest;
        Some x
```

OCaml ↗

to be
or not None

Between two calls the iterator remembers where it is in the list, in a variable of its own, `here`. It is the **counter** of slide 10: a closure with a state.

Generators

How can the producer remember where it stopped?

A generator, in JavaScript

```
function* two() {  
  yield 'h';  
  yield 'i';  
}  
  
const g = two();  
console.log(g.next(), g.next(), g.next());
```

JAVASCRIPT

Run

Reset

What do the three calls of `next` answer?

`yield` gives one element and suspends the function. The next call of `next` resumes it after that `yield`.

The answer has two fields: `value`, the element, and `done`, which is true when the function has ended.

```
{ value: "h", done: false } { value: "i", done: false } { value: undefined, done: true }
```

A generator is an **iterator** that we write as a function: the language remembers where it stopped.

The object the compiler makes of a generator

```
function* two() {  
  yield 'h';  
  yield 'i';  
}
```

JAVASCRIPT

Each **yield** is one **case**: it writes in **pc** where to resume, and returns the element.

After the last **yield**, **done** is true.

```
const two = () => {  
  let pc = 0; // where next() resumes  
  const next = () => {  
    switch (pc) {  
      case 0: pc = 1; return { value: 'h', done: false }; // yield 'h'  
      case 1: pc = 2; return { value: 'i', done: false }; // yield 'i'  
      default: return { value: undefined, done: true }; // the end  
    }  
  };  
  return { next };  
};  
  
const g = two();  
console.log(g.next(), g.next(), g.next());
```

JAVASCRIPT

Run

Reset

```
{ value: "h", done: false } { value: "i", done: false } { value: undefined, done: true }
```

From inside, a generator is an **object**: its local variables, and a position in its code.

Pull and push

	pull: the consumer asks	push: the producer calls
many values	an iterator: we call <code>next</code>	a function passed to <code>iter</code> : it is called once per element
one value	block 2	session 6

In pull, the collection gives us an iterator and we call it. In push, we give the collection a function and it calls it.

External iteration is pull, internal iteration is push. With pull, **we** choose when to ask for the next element, and we stop by not asking.

Source: Meijer 2010, *Subject/Observer is dual to Iterator*; the table follows the Rx design notes.

Conclusion of block 1

the question

the answer

- | | | |
|---|---|---|
| 1 | What does a function value carry? | Its code and its environment: a closure . |
| 2 | How do we visit every element of a list? | We pass a function to a fold: internal iteration . |
| 3 | Can our function stop the traversal? | By its answer, only with <code>fold_until</code> : <code>Continue</code> or <code>Stop</code> . |
| 4 | How can the consumer choose the next step? | It calls <code>next</code> : external iteration . |
| 5 | How can the producer remember where it stopped? | With <code>yield</code> : a generator , which the language suspends. |

In this block a function carries out **one step of a traversal**; we asked who calls it: the collection, or us.

In the next block a function holds **a computation not yet run**, and we ask when to run it.